

Lecture 6: Leases and LBFS (Reducing Communication)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison
Spring 2025

Today's Topic: Leases and LBFS

Leases - Cary G. Gray and David R. Cheriton, Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency, Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP), December 1989, **SIGOPS Hall of Fame Award**

LBFS Athicha Muthitacharoen, Benjie Chen (MIT), David Mazieres (NYU), A Low-Bandwidth Network File System, SOSP'01

Why Today's Papers?

- Leases:
 - Useful for delivering cache consistency in presence of host and communication failures
 - Simple mechanism: Assumes physical clocks
 - Used in many systems
- LBFS:
 - Good summary of related work (NFS, AFS, Coda, Leases)
 - Neat techniques (used in storage deduplication too): Hashes and fingerprints
 - Reasonable design decisions building on top of NFS/AFS

Motivation for Leases

Node thinks it has “authority” for some action, but something fails

- Network partition or crash

Authority:

- Cache: serve, or modify data
- Control metadata: directory contents, placement, ownership
- Coordination: leader of consensus algorithm

Problem:

Authority easy when all nodes alive + connected, difficult under failure

Hard to continue progress and remain available

Goal: Time-Bounded Authority

Mechanism	Authority	Failure behavior
Locks	Explicit, revocable	Can block forever
Callbacks	Explicit, revocable	Requires connectivity
Leases	Implicit, time-limited	Authority expires safely

Other approaches for authority don't behave well under failure

Lease is promise that *time* will clean up after failures

Lease: Consistent Caching on Clients

Lease:

- Contract that gives its holder specified rights for a limited period of time

Term:

- Time period holder has control over writes;
Server must get approval from holder for writes during term

Steps for client to read into local cache:

- Client fetches data from server and obtains lease for some term
- Future reads can remain local during term (guaranteed to have up-to-date consistent copy)
- Client asked for approval of any writes on server during term

Client writes back to server:

- If no clients have lease, server can write
- Otherwise, server gets approval from all lease holders (or waits until leases expire)

Leases: Renewals

Lease renewal:

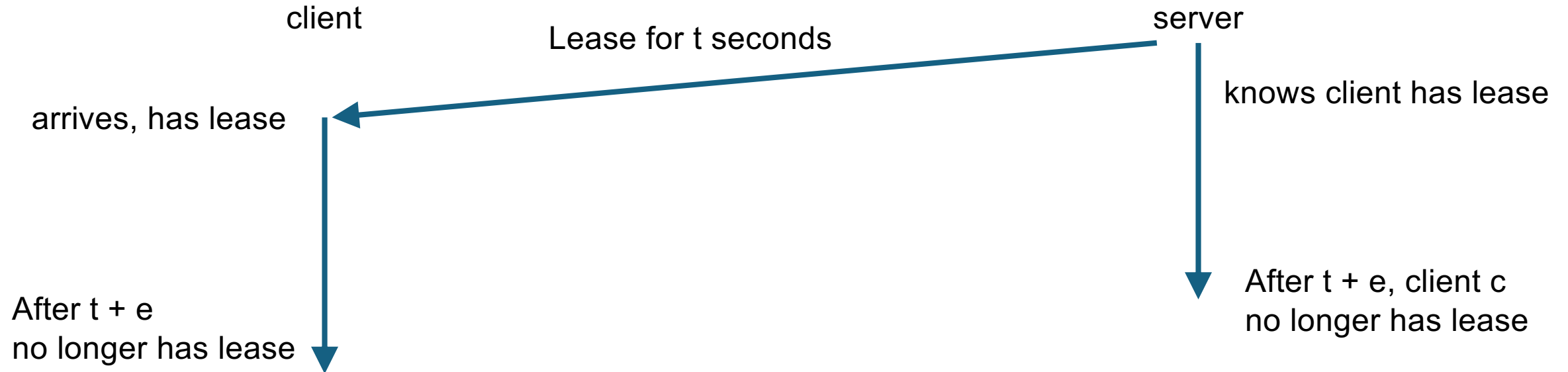
- Client asks server to extend lease before it expires

If lease will soon expire, should client request extension or not?

- Yes:
- No:
- Worst case:

Leases: Physical Clocks

Term t



Implications

- 1.
- 2.
- 3.
- 4.

Lease: Term Length can be Adaptive

Advantages of Short Lease Terms

Advantages of Long Lease Terms

Leases: How to set Term?

Heavily Write-shared file:

- Lease term =
- Why?

Distant client:

- Lease term =
- Why?

Leases: Two Extremes of AFS

Version 1: Poll on every file open

Lease term =

Problem?

Version 2: Callbacks on server – notify client of file changes

Lease term =

Problem?

Alternative?

Leases: Optimizations for “Installed” Files

File Properties: Widely shared, heavily read, infrequently written

Use small number of leases across multiple files

Server multi-casts extension to all clients periodically; **why good idea?**

-

What to do when file is written?

-

When can write proceed?

-

- Is this longer latency than asking clients for approval?

-

Advantage for server state?

-

Leases: Summary

Basic idea

- Authority over resource while have lease; must be asked to accept changes while hold lease
- Passing of time automatically handles failures

Trade-offs: Determining best lease term

- **Too long**
 - False sharing
 - Must wait longer after failure
- **Too short**
 - Not safe if doesn't account for network latency
 - Overhead of continued renewals
- Adaptive: Set lease term depending on workload, environment

Advantages

- Handles host and communication failures
- Scales well, easy fast path
- Simple mechanism + assumptions
 - Just physical clocks, network latency $\ll$ term

Low-Bandwidth Network File System (LBFS) - 2001

Motivation?

-

Goal: Metric?

-

- Not?

- Related technique?

-

Workload Assumptions?

Related Work: Reduce Network Traffic?

AFS:

-

Leases: Lease terms

-

NFSv4:

-

Coda:

LBFS: Basic Idea

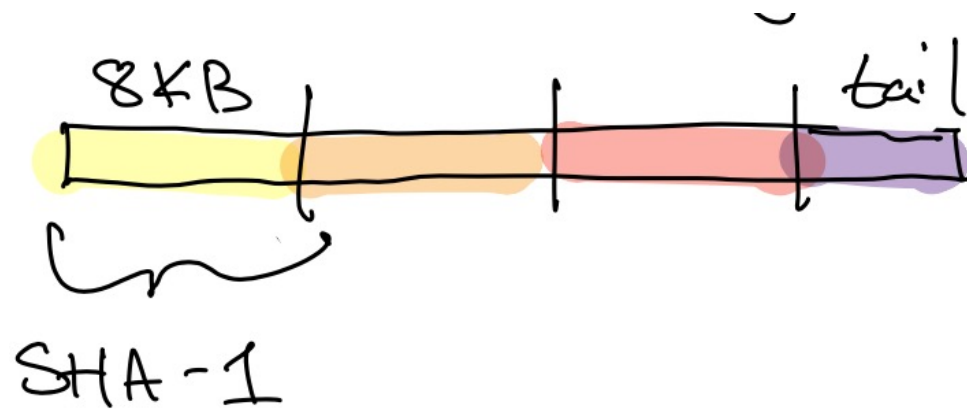
Don't send data chunks over network if client/server already has it

How to identify unique chunks?

Trade-off in identifying identical chunks?

-
-

Option 1: Aligned, non-overlapping chunks



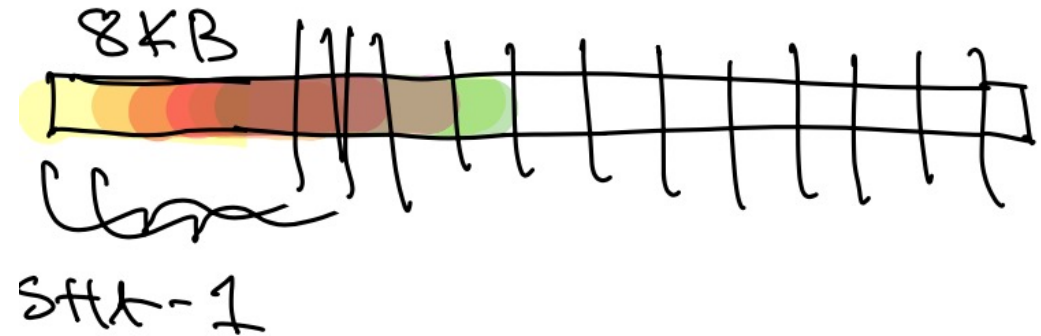
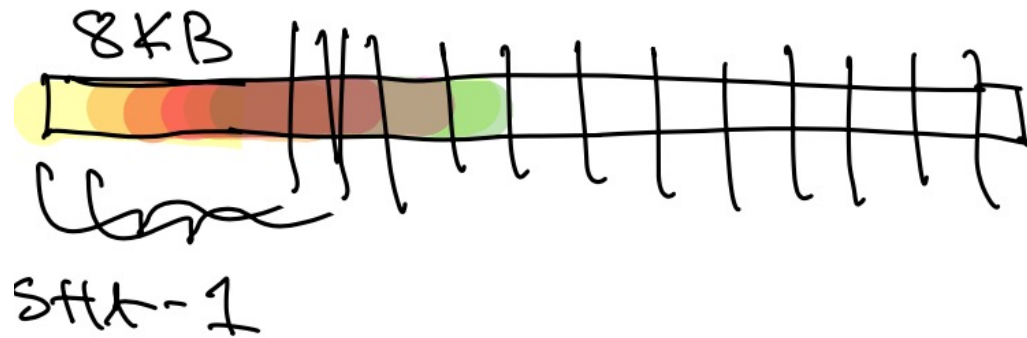
What will it find?

What won't it find?

Pros:

Cons:

Option 2: Sliding-Window of Overlapping Chunks

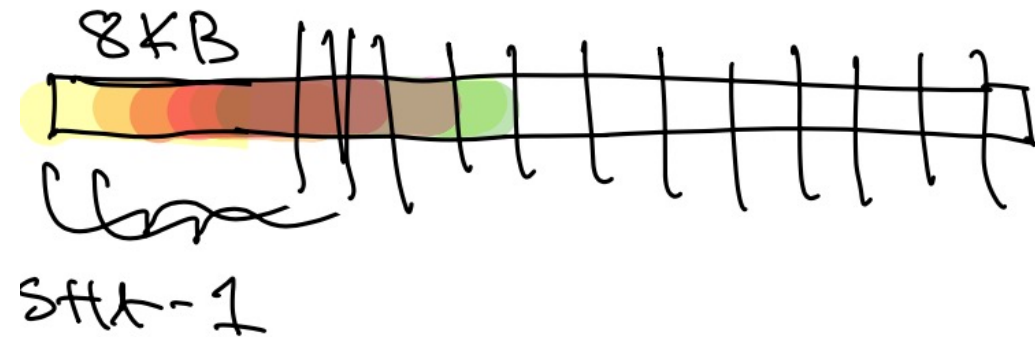
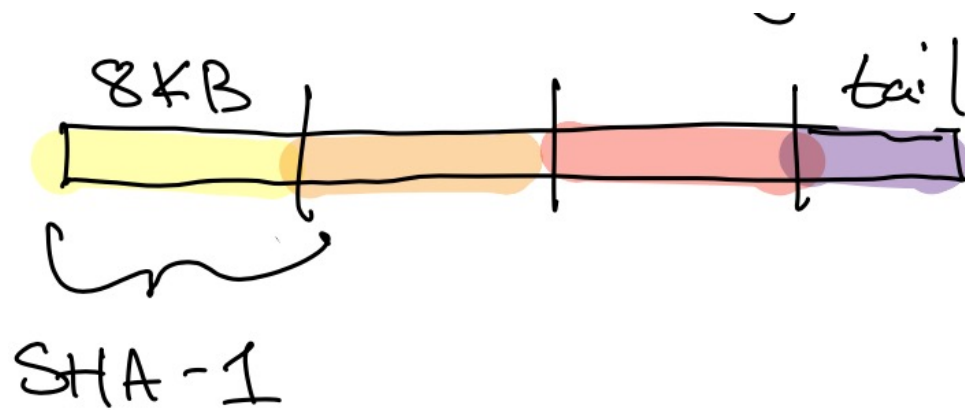


Pros:

Cons:

Option 3: Related Work – rsync

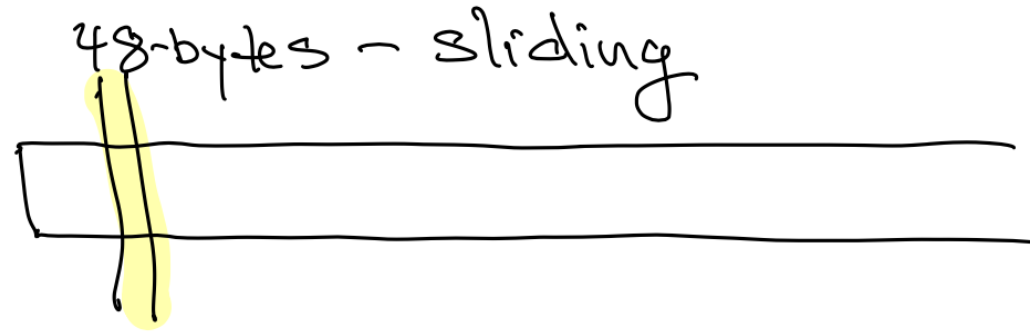
Combination



Limitations?

-

LBFS Chunk? Not Fixed Size



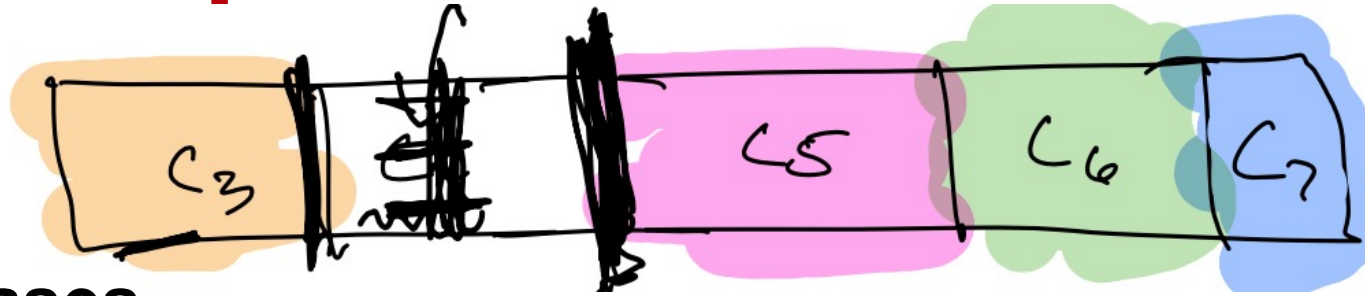
Compute Rabin's fingerprint over 48-bytes

$$R = \boxed{\text{---}} \mid \underbrace{\text{---}}_{13 \text{ bits}} \Rightarrow \text{Breakpoint?}$$

Probable size of each chunk?

Represent each chunk?

LBFS: Properties w/ Rabin's Fingerprint?



Common cases

- Add data to c_4 ? Which chunks change?
- Delete data from c_4 ?

Less common

- Add data that matches fingerprint in c_4 ?
- Delete data was fingerprint at end of c_4 ?

Practical Issues

- Too small or too large chunks (all zeros) $\rightarrow$ define min and max chunksize
- Suppress breakpoint if $< \text{min}$; force breakpoint if $> \text{max}$

LBFS: Chunk Database on Client & Server

Map 64-bit key (from 160 bit SHA-1 hash) $\leftrightarrow$ (file, offset, count)

Update on all file changes

What if file modified outside of LBFS?

Implication?

LBFS Cache Consistency?

AFS with Leases

Avoids unnecessary traffic w/ server

Why whole file?

What to do on open?

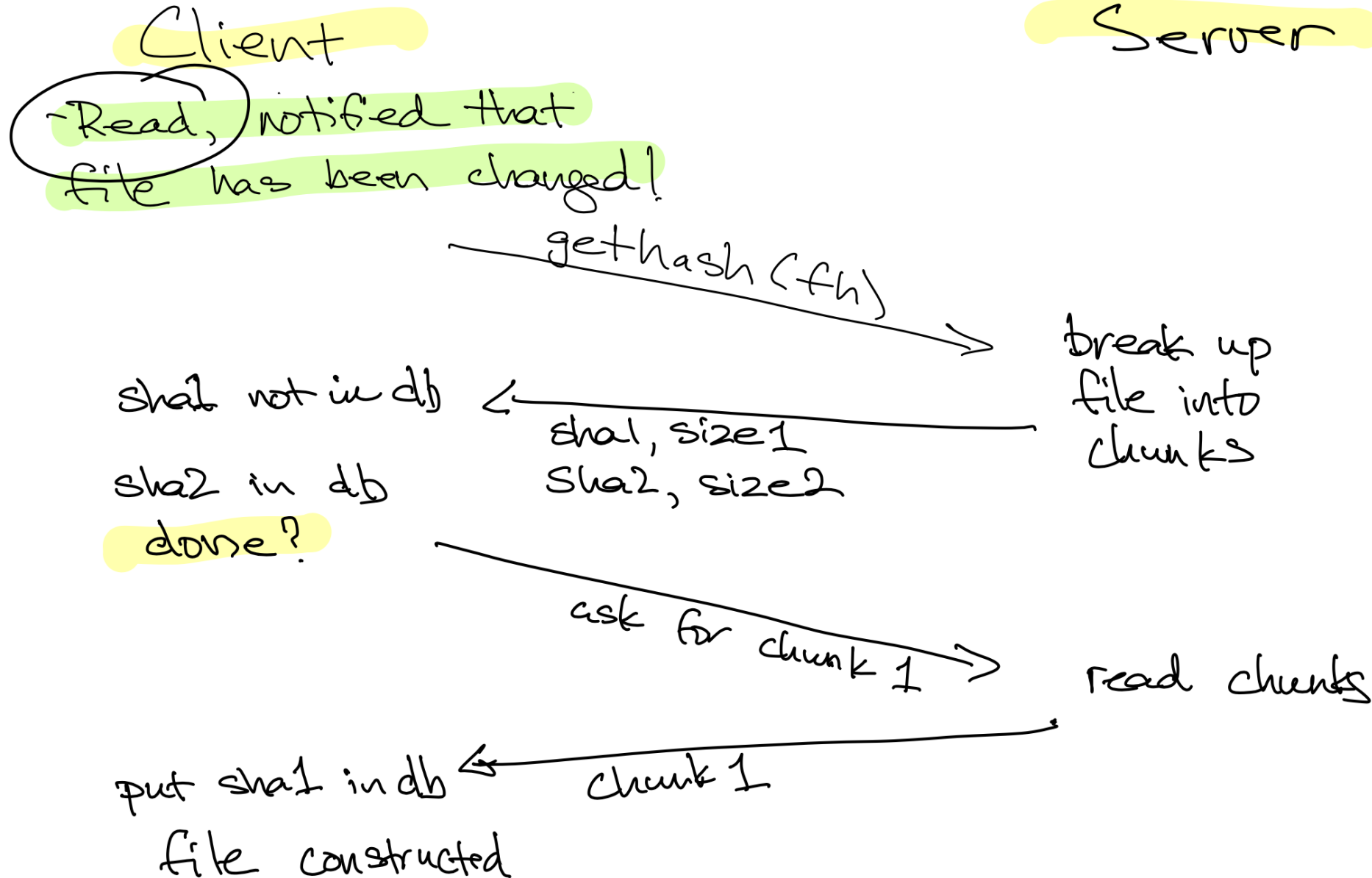
If still have lease,

If lost lease,

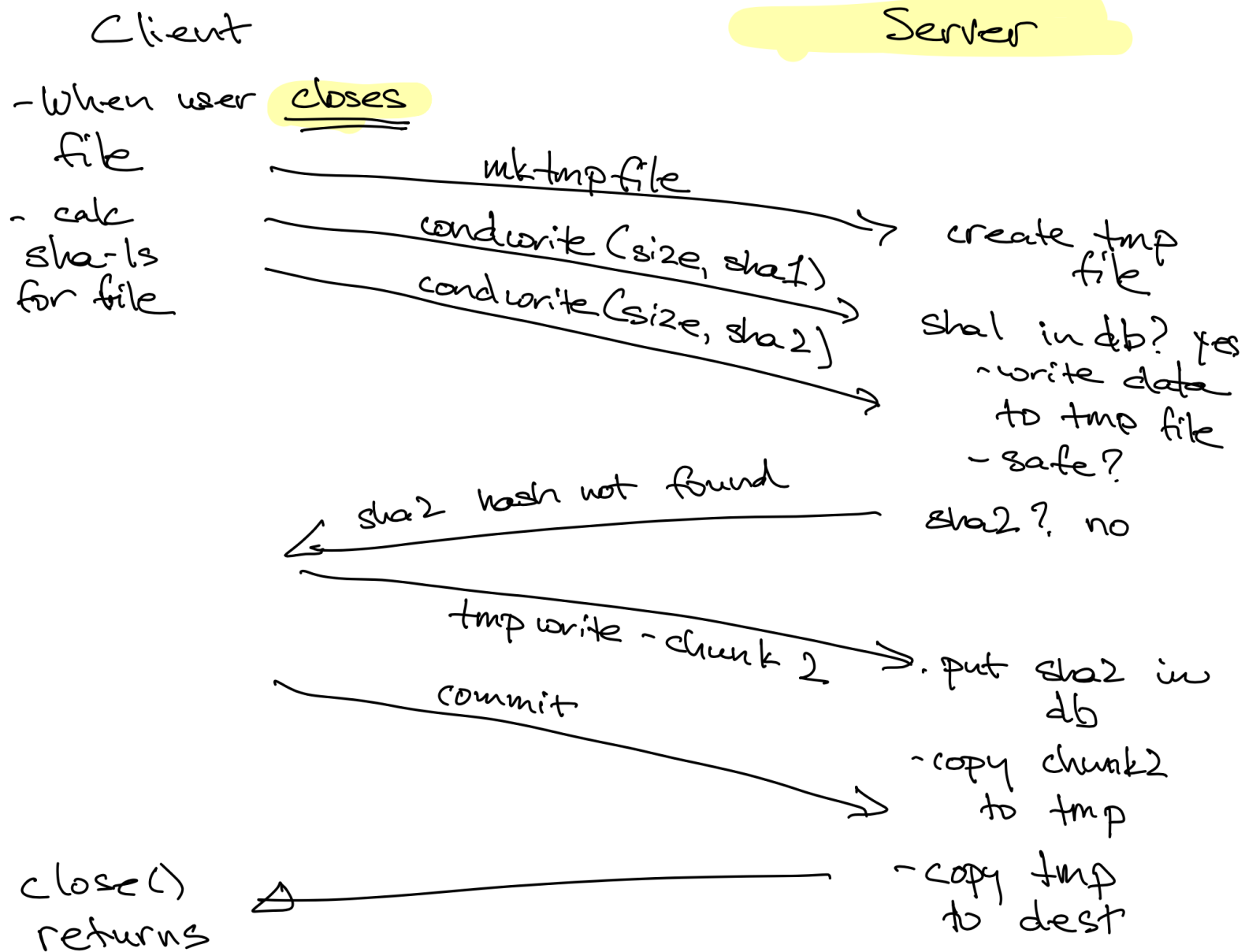
If file is different,

What to do on close?

LBFS: Open/Read Protocol



LBFS: Close/Write Protocol



Ever worse than without?

What does server do at end?

Problem with
Unix filesystem
semantics?

Summary of LBFS

Minimize network traffic

LBFS: Only sends data other side doesn't already have

- Assumes small changes to files
- Use SHA-1 hash to identify/name chunks
- Rabin fingerprints: Break data up into variable-sized chunks; data changes impact minimal chunks
- Chunk database: Only hints! Can't be trusted since files can be modified outside of LBFS

Announcements

Reading Group Week 3: New groups, Due Sunday evening

Mini-Project 1: RPC for Client/Server KV Store – Due this Sunday

- Signup for 10-minute demo w/ TA – Monday, Tuesday, Wednesday

Mini-Project 2: Available early next week

- Extend Mini-Project 1:
Add durability and key-space partitioning ACROSS independent servers
- Can work w/ same project partner or different one

Friday:

- **Ordering -- Logical Clocks and Distributed Snapshots**